L11: Policy Gradient Methods

EECE 576N | From MDPs to Deep RL | Fall 2026

Cyrus Neary | cyrus.neary@ubc.ca

So far…

$Q(s, a)$

Solve for (optimal) value function

- Dynamic programming
  - Value iteration
  - Policy iteration
- Tabular learning methods
  - Monte Carlo estimation
  - TD Learning
- Function approximation
  - Deep Q-Learning



$\pi(a \mid s)$

Construct a policy in terms of the approximate value function

- Argmax over actions
  - $\pi(s) = \arg\max_{a \in A} Q(s, a)$
- Epsilon-greedy policies
  - $\pi(a \mid s) = \begin{cases} \arg\max_{a \in A} Q(s, a), & \text{w.p. } 1 - \epsilon \\ \text{Uniform}(A), & \text{w.g. } \epsilon \end{cases}$

So far…

$Q(s, a)$

Solve for (optimal) value function

- Dynamic programming
  - Value iteration
  - Policy iteration
- Tabular learning methods
  - Monte Carlo estimation
  - TD Learning
- Function approximation
  - Deep Q-Learning



$\pi(a \mid s)$

Construct a policy in terms of the approximate value function

- Argmax over actions
  - $\pi(s) = \arg\max_{a \in A} Q(s, a)$
- Epsilon-greedy policies
  - $\pi(a \mid s) = \begin{cases} \arg\max_{a \in A} Q(s, a), & \text{w.p. } 1 - \epsilon \\ \text{Uniform}(A), & \text{w.g. } \epsilon \end{cases}$

Today: Parameterize the policy function $\pi_\theta(a \mid s)$.

Use gradient descent directly on the policy's parameters, to fine a local maximum of $\theta \in \mathbb{R}^d$ that maximizes our sequential decision-making objective.

The optimization objective

Given a parameterized policy function $\pi_\theta(a \mid s)$, we want to find $\theta \in \mathbb{R}^d$ that maximizes some measure of performance $J(\theta)$.

The optimization objective

Given a parameterized policy function $\pi_\theta(a \mid s)$, we want to find $\theta \in \mathbb{R}^d$ that maximizes some measure of performance $J(\theta)$.

When $J(\theta)$ a differentiable function that maps policy parameter vectors $\theta \in \mathbb{R}^d$ to scalar values, we can use gradient descent:

$$\theta_{t+1} = \theta_t + \alpha \nabla J(\theta)$$

The optimization objective

Given a parameterized policy function $\pi_\theta(a \mid s)$, we want to find $\theta \in \mathbb{R}^d$ that maximizes some measure of performance $J(\theta)$.

When $J(\theta)$ a differentiable function that maps policy parameter vectors $\theta \in \mathbb{R}^d$ to scalar values, we can use gradient descent:

$$\theta_{t+1} = \theta_t + \alpha \nabla J(\theta)$$

A common choice for this performance measure $J(\theta)$ is the expected return of trajectories τ under policy π_θ executed from some distribution ρ of initial states s_0:

$$J(\theta) = \mathbb{E}_{s_0 \sim \rho} \left[V^{\pi_\theta}(s_0) \right]$$

The optimization objective

Given a parameterized policy function $\pi_\theta(a \mid s)$, we want to find $\theta \in \mathbb{R}^d$ that maximizes some measure of performance $J(\theta)$.

When $J(\theta)$ a differentiable function that maps policy parameter vectors $\theta \in \mathbb{R}^d$ to scalar values, we can use gradient descent:

$$\theta_{t+1} = \theta_t + \alpha \nabla J(\theta)$$

A common choice for this performance measure $J(\theta)$ is the expected return of trajectories τ under policy π_θ executed from some distribution ρ of initial states s_0:

$$J(\theta) = \mathbb{E}_{s_0 \sim \rho}\left[V^{\pi_\theta}(s_0)\right]$$

In practice, we can't evaluate $J(\theta) = \mathbb{E}_{s_0 \sim \rho}\left[V^{\pi_\theta}(s_0)\right]$ exactly. Instead, we use estimates $\widehat{J(\theta)}$ to define our gradient steps.

$$\theta_{t+1} = \theta_t + \alpha \widehat{\nabla J(\theta)}$$

Common policy parameterizations

Discrete actions

A is a finite set $\Longrightarrow$ Categorical distribution over A.

E.g., softmax distribution: $\pi(a \mid s) = \dfrac{\exp\left(f_\theta(s,a)\right)}{\sum_{a' \in A} \exp\left(f_\theta(s,a')\right)}$

Where $f_\theta(s,a)$ is some parameterized function mapping state-action pairs to some numerical preference value.

Note: practically, this is typically implemented as a neural network that maps states s to a vector of *logits* $z_\theta(s) \in \mathbb{R}^{|A|}$.

Common policy parameterizations

Discrete actions

A is a finite set $\Longrightarrow$ Categorical distribution over A.

E.g., softmax distribution: $\pi(a \mid s) = \frac{\exp\left(f_\theta(s,a)\right)}{\sum_{a' \in A} \exp\left(f_\theta(s,a')\right)}$

Where $f_\theta(s,a)$ is some parameterized function mapping state-action pairs to some numerical preference value.

Note: practically, this is typically implemented as a neural network that maps states s to a vector of *logits* $z_\theta(s) \in \mathbb{R}^{|A|}$.

Continuous actions

A is a subset of $\mathbb{R}^m$ $\Longrightarrow$ Deterministic function + exploration noise.

E.g., parameterization of a deterministic mapping $\mu_\theta : S \to A$.

The parameterized policy could then be defined as $\pi_\theta(a_t \mid s_t) = \mu_\theta(s_t) + \eta_t$.

With $\eta_t \sim \mathcal{N}(0, \sigma^2 I)$ some externally-sampled exploration noise.

Common policy parameterizations

Discrete actions

A is a finite set $\Longrightarrow$ Categorical distribution over A.

E.g., softmax distribution: $\pi(a \mid s) = \frac{\exp\left(f_\theta(s,a)\right)}{\sum_{a' \in A} \exp\left(f_\theta(s,a')\right)}$

Where $f_\theta(s,a)$ is some parameterized function mapping state-action pairs to some numerical preference value.

Note: practically, this is typically implemented as a neural network that maps states s to a vector of *logits* $z_\theta(s) \in \mathbb{R}^{|A|}$.

Continuous actions

A is a subset of $\mathbb{R}^m$ $\Longrightarrow$ Deterministic function + exploration noise.

E.g., parameterization of a deterministic mapping $\mu_\theta : S \to A$.

The parameterized policy could then be defined as $\pi_\theta(a_t \mid s_t) = \mu_\theta(s_t) + \eta_t$.

With $\eta_t \sim \mathcal{N}(0, \sigma^2 I)$ some externally-sampled exploration noise.

A is a subset of $\mathbb{R}^m$ $\Longrightarrow$ Parameterized Gaussian distribution.

E.g., use neural networks to parameterize $\mu_\theta(s)$ and $\Sigma_\theta(s)$.

The stochastic policy is then given by $\pi(a \mid s) = \mathcal{N}(\mu_\theta(s), \Sigma_\theta(s))$.

Policy parameterization examples continued

RL-based finetuning of LLMs

A is the discrete set of tokens in a language model's vocabulary $\Longrightarrow$ policy is given by a language model itself.

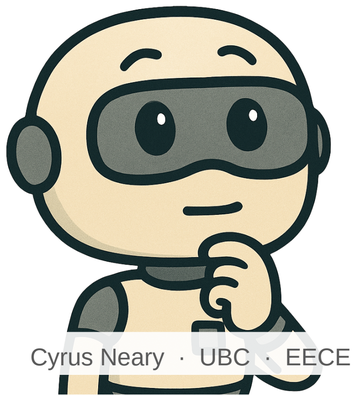
In language generation, state s_t is given by the tokens so far $x_{1:t}$ (context), and the action is given by the next token $a_t = x_{t+1}$.

We can think of the LLM as an RL policy: $\pi_\theta(a_t \mid s_t) = LLM_\theta(x_{t+1} \mid x_t)$
And train $\pi_\theta(a \mid s)$ using gradient descent algorithms.



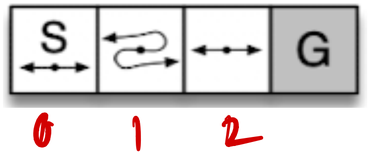
Why might policy gradient methods be advantageous over value-based methods?

- Value methods often use bootstrapping, can introduce problems (e.g. deadly triad)
  - "Pure" policy gradient methods don't necessarily have this issue.
- Directly parametrize policy functions in continuous/high-dimensional action spaces.
  - No need to explicitly solve $\arg\max_a Q(s,a)$.
- Naturally supports highly expressive, stochastic policies.
  - e.g. RL for training LLMs
- "Don't solve a harder problem as an intermediate step".
- Let's us impose structure on the policy itself.
  - we can sometimes impose priors on actions better than on values.



Example: the flexibility of policy parameterization

Consider the following four-state “corridor” environment.



Actions: “left” and “right”.

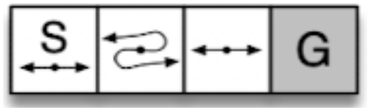
Dynamics:

- “left” and “right” result in the expected transitions in the first and third states,
- But result in the opposite transitions when taken from the second state.

Reward: -1 per step, unless the agent is in the terminal goal state, marked “G”.

Example: the flexibility of policy parameterization

Consider the following four-state “corridor” environment.



Actions: “left” and “right”.

Dynamics:
- “left” and “right” result in the expected transitions in the first and third states,
- But result in the opposite transitions when taken from the second state.

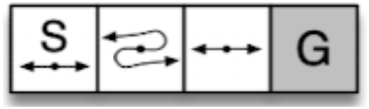
Reward: -1 per step, unless the agent is in the terminal goal state, marked “G”.

∀S

Suppose we parameterized the state-action value function linearly as $Q_\theta(s,a) = \theta^T x(s,a)$, with $x(s, \text{left}) = [1,0]^T$ and $x(s, \text{right}) = [0,1]^T$.

Example: the flexibility of policy parameterization

Consider the following four-state “corridor” environment.



Actions: “left” and “right”.

Dynamics:
- “left” and “right” result in the expected transitions in the first and third states,
- But result in the opposite transitions when taken from the second state.

Reward: -1 per step, unless the agent is in the terminal goal state, marked “G”.

Suppose we parameterized the state-action value function linearly as $Q_\theta(s,a) = \theta^T x(s,a)$, with $x(s, \text{left}) = [1,0]^T$ and $x(s, \text{right}) = [0,1]^T$.

Note: with this parameterization, $Q_\theta(s,a)$ cannot differentiate between the states. This means any ϵ-greedy policy w.r.t. Q_θ will either need to go right with high probability, or go left with high probability.

On the other hand, the optimal stochastic policy assigns almost equal probability to the two actions. This can be achieved with softmax policy parameterizations, but not with ϵ -greedy value-based policies.

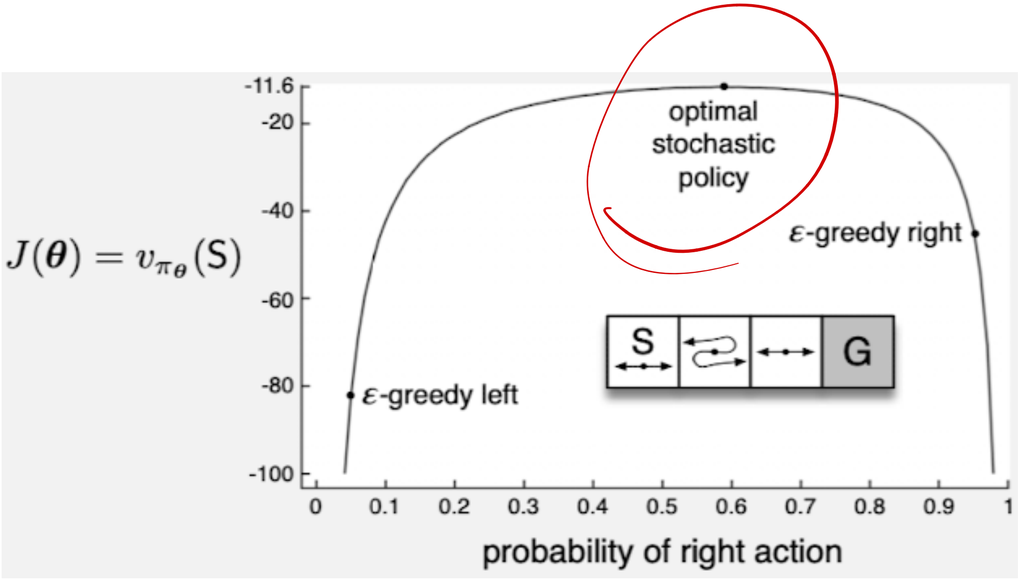


Figure courtesy of Sutton and Barto (2018)

How are policy gradient algorithms possible without a model of the environment?

Recall! We want to compute $\nabla J(\theta) = \nabla \mathbb{E}[V^{\pi_\theta}(s_0) \mid s_0 \sim \rho] = \nabla \mathbb{E}\left[\sum_{t=0}^{T} R_t \mid \pi_\theta, s_0\right]$

$$\nabla J(\theta) = \nabla \sum_{\tau} P(\tau|\theta) R(\tau)$$

$$\hookrightarrow P(\tau|\theta) = \rho(s_0) \prod_{t=0}^{T-1} \pi_\theta(a_t|s_t) T(s_{t+1}|s_t, a_t)$$

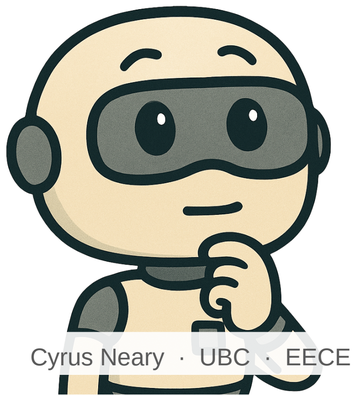
Complicated! The prob of $P(\tau|\theta)$ depends on nested compositions of π_θ and T.

$\theta \rightarrow \pi_\theta(a_t|s_t) \rightarrow a_t \rightarrow s_{t+1} \sim T(\cdot|a_t, s_t) \rightarrow P(\tau, \theta)$

If we naively apply chain rule along sampled trajectories, our grad would appear to depend on $T(\cdot|s,a)$.

↳ In RL, we don't this!

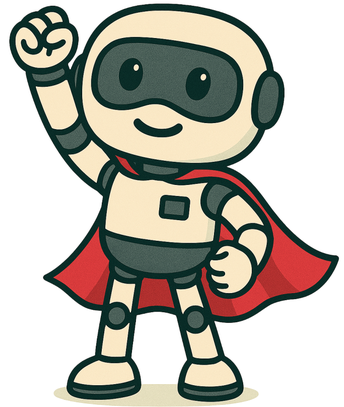
How do we compute $\nabla J(\theta)$?



The policy gradient theorem! To the rescue!

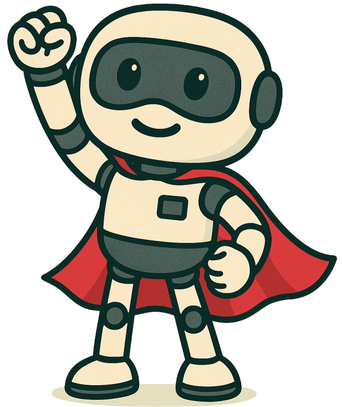
$$\nabla J(\theta) = \nabla V^{\pi}(s_0)$$

$$\propto \sum_{s \in S} \mu(s) \sum_{a \in A} Q_{\pi_\theta}(s, a) \nabla \pi_\theta(s, a)$$



The policy gradient theorem! To the rescue!

$$\nabla J(\theta) = \nabla V^{\pi}(s_0)$$
$$\propto \sum_{s \in S} \mu(s) \sum_{a \in A} Q_{\pi_\theta}(s, a) \nabla \pi_\theta(s, a)$$



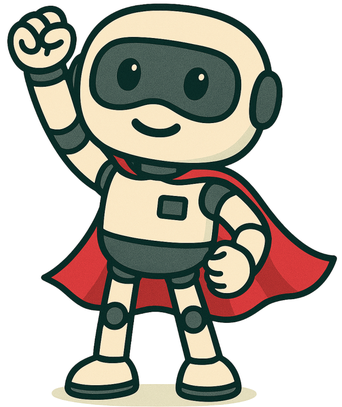
Normalized expected state visitation frequency over an episode induced by policy π_θ.

Note: this part will be slightly different in different cases (episodic vs. Continuing, discounted vs undiscounted vs average reward).

The policy gradient theorem! To the rescue!

$$\nabla J(\theta) = \nabla V^{\pi}(s_0)$$

$$\propto \sum_{s \in S} \mu(s) \sum_{a \in A} Q_{\pi_\theta}(s,a) \nabla_{\theta} \pi_\theta(s,a)$$



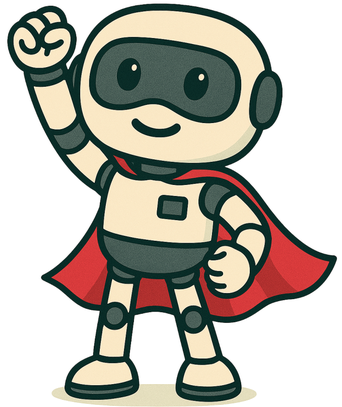
Normalized expected state visitation frequency over an episode induced by policy π_θ.

Note: this part will be slightly different in different cases (episodic vs. Continuing, discounted vs undiscounted vs average reward).

True state-action value function of policy π_θ.

The policy gradient theorem! To the rescue!

$$\nabla J(\theta) = \nabla V^{\pi}(s_0)$$
$$\propto \sum_{s \in S} \mu(s) \sum_{a \in A} Q_{\pi_\theta}(s,a) \nabla \pi_\theta(s,a)$$



Normalized expected state visitation frequency over an episode induced by policy π_θ.

Note: this part will be slightly different in different cases (episodic vs. Continuing, discounted vs undiscounted vs average reward).

True state-action value function of policy π_θ.

Crucially: $\nabla J(\theta)$ doesn't depend on the gradient through the environment dynamics!

Algorithmic implication: We can estimate $\nabla J(\theta) = \mathbb{E}_{S \sim \mu, A \sim \pi_\theta(\cdot|S)} \left[Q_{\pi_\theta}(S,A) \nabla log(\pi_\theta(A \mid S)) \right]$ by running π_θ in the environment, estimating $Q_{\pi_\theta}(s,a)$, and computing averages of samples of $Q^{\pi_\theta}(s^{(i)}, a^{(i)}) \nabla \log(\pi_\theta(s^{(i)}, a^{(i)}))$.

Proving the policy gradient theorem for undiscounted rewards

Undiscounted episodic case $\quad J(\theta) = \mathbb{E}[V^{\pi_\theta}(s_0) \mid s_0 \sim \rho]$

Consider $\nabla V^{\pi_\theta}(s)$ for some initial state s.

$$\nabla V^{\pi_\theta}(s) = \nabla \Big[\sum_a \pi_\theta(a|s)\, Q^{\pi_\theta}(s,a) \Big] \text{ by def of } V^\pi \text{ and } Q^\pi.$$

Apply product rule to RHS.

$$\nabla V^{\pi_\theta}(s) = \sum_a \Big\{ Q^{\pi_\theta}(s,a) \nabla \pi_\theta(a|s) + \pi_\theta(a|s) \nabla Q^{\pi_\theta}(s,a) \Big\}$$

$$= \sum_a \Big\{ \qquad\qquad + \pi_\theta(a|s) \nabla \Big[\sum_{s',r} T(s'|s,a)\,[r + V^{\pi_\theta}(s')] \Big] \Big\}$$

$$= \sum_a \Big\{ \qquad\qquad + \pi_\theta(a|s) \Big[\sum_{s',r} T(s'|s,a) \big(\nabla r^{\,0} + \nabla V^{\pi_\theta}(s') \big) \Big] \Big\}$$

Define $\sigma(s) := \sum_a Q^{\pi_\theta}(a,s) \nabla \pi_\theta(a|s)$

$$\nabla V^{\pi_\theta}(s) = \sigma(s) + \sum_a \pi_\theta(a|s) \sum_{s',r} T(s'|s,a) \nabla V^{\pi_\theta}(s')$$

Proving the policy gradient theorem for undiscounted rewards

$$\nabla V^{\pi_\theta}(s) = \sigma(s) + \sum_a \pi_\theta(a|s) \sum_{s',r} T(s'|s,a) \nabla V^{\pi_\theta}(s')$$

$$\nabla V^{\pi_\theta}(s') = \sigma(s') + \sum_{a'} \sum_{s''} \pi(a'|s') T(s''|s',a') \nabla V^{\pi_\theta}(s'')$$

$$\nabla V^{\pi_\theta}(s) = \sigma(s) + \sum_a \sum_{s'} \pi_\theta(a|s) T(s'|s,a) \Big\{ \sigma(s') + \sum_{a'} \sum_{s''} \pi(a'|s') T(s''|s',a') \nabla V^{\pi_\theta}(s'') \Big\}$$

$$\sum_a \sum_{s'} \pi_\theta(a|s) T(s'|s,a) = \sum_{s^{next}} P(s \to s^{next}, 1, \theta)$$

Probability of transitioning from s to s' in 1 step under π_θ

$$\nabla V^{\pi_\theta}(s) = \sigma(s) + \sum_{s'} P(s \to s', 1, \theta) \Big\{ \sigma(s') + \sum_{s''} P(s' \to s'', 1, \theta) \nabla V^{\pi_\theta}(s'') \Big\}$$

$$= \sigma(s) + \sum_{s'} P(s \to s', 1, \theta) \sigma(s') + \sum_{s''} \sum_{s'} P(s \to s', 1, \theta) P(s' \to s'', 1, \theta) \nabla V^{\pi_\theta}(s'')$$

$P(s \to s'', 2, \theta)$

Proving the policy gradient theorem for undiscounted rewards

$$\nabla V^{\pi_\theta}(s) = \sigma(s) + \sum_{s'} P(s \to s', 1, \theta)\sigma(s')$$
$$+ \sum_{s''} P(s \to s'', 2, \theta)\sigma(s'')$$
$$+ \sum_{s'''} P(s \to s''', 3, \theta)\sigma(s''') + \cdots$$
$$= \sum_{x \in S} \sum_{k=0}^{\infty} P(s \to x, k, \theta)\sigma(x)$$

$$\nabla V^{\pi_\theta}(s) = \sum_{x \in S} \sum_{k=0}^{\infty} P(s \to x, k, \theta) \sum_{a} Q^{\pi_\theta}(x, a) \nabla \pi_\theta(a|x)$$

Thus $\nabla \mathbb{E}[V^{\pi_\theta}(s_0) | s_0 \sim \rho] = \sum_{s_0 \in S} \rho(s_0) \nabla V^{\pi_\theta}(s_0)$

$$= \sum_{s_0} \rho(s_0) \sum_{s \in S} \sum_{k=0}^{\infty} P(s_0 \to s, k, \theta) \sum_{a} Q^{\pi_\theta}(s, a) \nabla \pi_\theta(a|s)$$

$$= \sum_{s} \sum_{s_0} \rho(s_0) \sum_{k=0}^{\infty} P(s_0 \to s, k, \theta) \sum_{a} Q^{\pi_\theta}(s, a) \nabla \pi_\theta(a|s)$$

?

Proving the policy gradient theorem for undiscounted rewards

$$= \sum_s \underbrace{\sum_{s_0} \rho(s_0) \sum_{k=0}^{\infty} P(s_0 \to s, k, \theta)}_{\eta(s) = \sum_{s_0} \rho(s_0) \sum_{k=0}^{\infty} P(s_0 \to s, k, \theta)} \sum_a Q^{\pi_\theta}(s,a) \nabla \pi_\theta(a|s)$$

$\eta(s)$: the expected number of visits to state s during an episode starting from $s_0 \sim \rho$, and following π_θ.

$\eta(s)$ is not a distribution.

$$\nabla J(\theta) = \sum_s \eta(s) \sum_a Q^{\pi_\theta}(s,a) \nabla \pi_\theta(a|s)$$

Consider normalizing $\eta(s)$, $\mu(s) = \frac{\eta(s)}{\sum_{s'} \eta(s')}$

This is the normalized occupancy, which tells us the probability of being in state s at some random time within an episode under π_θ

Then $$\nabla J(\theta) = \sum_{s'} \eta(s') \sum_s \frac{\eta(s)}{\sum_{s'} \eta(s')} \sum_a Q^{\pi_\theta}(s,a) \nabla \pi_\theta(a|s)$$

Proving the policy gradient theorem for undiscounted rewards

$$\nabla J(\theta) = \sum_{s'} \eta(s') \sum_{s} \frac{\eta(s)}{\sum_{s'} \eta(s')} \sum_{a} Q^{\pi_\theta}(s,a) \nabla \pi_\theta(a|s)$$

constant

$$\sum_{s} \mu(s)$$

$$\nabla J(\theta) \propto \sum_{s} \mu(s) \left[\sum_{a} Q^{\pi_\theta}(s,a) \nabla \pi_\theta(a|s) \right]$$

$$= \mathbb{E}_{s \sim \mu} \left[\sum_{a} Q^{\pi_\theta}(s,a) \nabla \pi_\theta(a|s) \right]$$

Hooray!!

Policy gradient theorem, continued

$$\begin{aligned}\nabla J(\theta) &= \nabla V^{\pi}(s_0) \\ &\propto \sum_{s \in S} \mu(s) \sum_{a \in A} Q_{\pi_\theta}(s, a) \nabla \pi_\theta(s, a)\end{aligned}$$

Policy gradient theorem, continued

$$\nabla J(\theta) = \nabla V^{\pi}(s_0)$$

$$\propto \sum_{s \in S} \mu(s) \sum_{a \in A} Q_{\pi_\theta}(s, a) \nabla \pi_\theta(s, a)$$

$$= \mathbb{E}_{S \sim \mu} \left[\sum_{a \in A} Q^{\pi_\theta}(S, a) \nabla \pi_\theta(a \mid S) \right]$$

Recall, $\mu(s)$ is the probability of being in state s at a random timestep within an episode starting from $s_0 \sim \rho$ and following policy π_θ.

Policy gradient theorem, continued

$$\nabla J(\theta) = \nabla V^{\pi}(s_0)$$

$$\propto \sum_{s \in S} \mu(s) \sum_{a \in A} Q_{\pi_\theta}(s, a) \nabla \pi_\theta(s, a)$$

$$= \mathbb{E}_{S \sim \mu} \left[\sum_{a \in A} Q^{\pi_\theta}(S, a) \nabla \pi_\theta(a \mid S) \right]$$

Recall, $\mu(s)$ is the probability of being in state s at a random timestep within an episode starting from $s_0 \sim \rho$ and following policy π_θ.

$$= \mathbb{E}_{S \sim \mu} \left[\sum_{a \in A} \pi_\theta(a \mid S) Q^{\pi_\theta}(S, a) \frac{\nabla \pi_\theta(a \mid S)}{\pi_\theta(a \mid S)} \right] = \mathbb{E}_{S \sim \mu} \left[\sum_{a \in A} \pi_\theta(a \mid S) Q^{\pi_\theta}(S, a) \nabla \ln(\pi_\theta(a \mid S)) \right]$$

Policy gradient theorem, continued

$$\nabla J(\theta) = \nabla V^{\pi}(s_0)$$

$$\propto \sum_{s \in S} \mu(s) \sum_{a \in A} Q_{\pi_\theta}(s, a) \nabla \pi_\theta(s, a)$$

$$= \mathbb{E}_{S \sim \mu} \left[\sum_{a \in A} Q^{\pi_\theta}(S, a) \nabla \pi_\theta(a \mid S) \right]$$

Recall, $\mu(s)$ is the probability of being in state s at a random timestep within an episode starting from $s_0 \sim \rho$ and following policy π_θ.

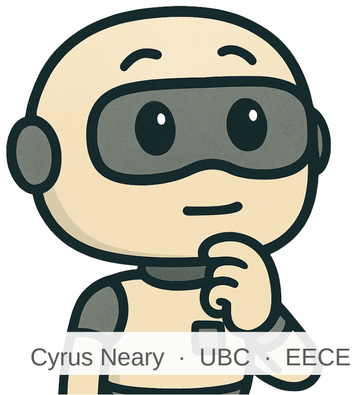
$$= \mathbb{E}_{S \sim \mu} \left[\sum_{a \in A} \pi_\theta(a \mid S) Q^{\pi_\theta}(S, a) \frac{\nabla \pi_\theta(a \mid S)}{\pi_\theta(a \mid S)} \right] = \mathbb{E}_{S \sim \mu} \left[\sum_{a \in A} \pi_\theta(a \mid S) Q^{\pi_\theta}(S, a) \nabla \ln(\pi_\theta(a \mid S)) \right]$$

$$= \mathbb{E}_{S \sim \mu,\, A \sim \pi_\theta(\cdot|S)} \left[Q^{\pi_\theta}(S, A) \nabla \ln(\pi_\theta(A \mid S)) \right]$$

A common form for the policy gradient theorem in papers.

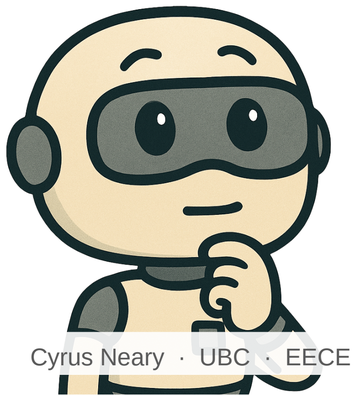
Naively, $\nabla J(\theta)$ seems like it should require differentiating through the dynamics $T(s' \mid s, a)$. Why does the policy gradient theorem let us avoid this?

A: It assumes T is deterministic, so its gradient is zero.
B: The visitation distribution $\mu(s)$ doesn't depend on θ, so we can ignore it.
C: The gradient becomes an expectation over states and actions visited by π_θ, and the only derivative inside is $\nabla \ln \left(\pi_\theta(a \mid s)\right)$, which we can estimate by running the policy.
D: We learn a model of T alongside the policy and differentiate through that.
E: I don't understand.



For a 7-joint robot arm with continuous torque commands, why is a Gaussian policy $\pi_\theta(a \mid s) = \mathcal{N}(\mu_\theta(s), \Sigma_\theta(s))$ more convenient than acting greedily wrt a learned $Q(s, a)$?

A: Gaussian policies always produce lower-variance gradients.
B: Acting only requires sampling from the Gaussian rather than solving an optimization problem.
C: Gaussian policies are guaranteed to contain the optimal policy.
D: Value functions can’t be defined over continuous action spaces.
E: I don’t understand.



The REINFORCE algorithm

$$\nabla J(\theta) \propto \mathbb{E}_{S\sim\mu,\, A\sim\pi_\theta(\cdot|S)} \left[Q^{\pi_\theta}(S, A) \nabla \ln(\pi_\theta(A \mid S)) \right]$$

The policy gradient theorem for the undiscounted, episodic case.

Note: we can’t evaluate the r.h.s exactly, but we can estimate it from samples.

The REINFORCE algorithm

$$\nabla J(\theta) \propto \mathbb{E}_{S\sim\mu,\, A\sim\pi_\theta(\cdot|S)} \left[Q^{\pi_\theta}(S, A) \nabla \ln(\pi_\theta(A \mid S)) \right]$$

The policy gradient theorem for the undiscounted, episodic case.

Note: we can't evaluate the r.h.s exactly, but we can estimate it from samples.

Idea: Rollout $\pi_\theta(s, a)$ in the environment to sample random trajectories $\tau = \left(S_0, A_0, R_0, S_1, A_1, R_1, \ldots, S_T\right)$.

The REINFORCE algorithm

$$\nabla J(\theta) \propto \mathbb{E}_{S\sim\mu,\, A\sim\pi_\theta(\cdot|S)} \left[Q^{\pi_\theta}(S, A) \nabla \ln(\pi_\theta(A \mid S)) \right]$$

The policy gradient theorem for the undiscounted, episodic case.

Note: we can't evaluate the r.h.s exactly, but we can estimate it from samples.

Idea: Rollout $\pi_\theta(s, a)$ in the environment to sample random trajectories $\tau = \left(S_0, A_0, R_0, S_1, A_1, R_1, \ldots, S_T\right)$.

Replace the exact state-action value function $Q^{\pi_\theta}(s, a)$ with a Monte Carlo approximation G_t .

Recall $Q^{\pi_\theta}(s, a) = \mathbb{E}\left[G_t \mid S_t = s, \pi_\theta\right]$

The REINFORCE algorithm

$$\nabla J(\theta) \propto \mathbb{E}_{S\sim\mu,\, A\sim\pi_\theta(\cdot|S)} \left[Q^{\pi_\theta}(S, A) \nabla \ln(\pi_\theta(A \mid S)) \right]$$

The policy gradient theorem for the undiscounted, episodic case.

Note: we can't evaluate the r.h.s exactly, but we can estimate it from samples.

Idea: Rollout $\pi_\theta(s, a)$ in the environment to sample random trajectories $\tau = \left(S_0, A_0, R_0, S_1, A_1, R_1, \ldots, S_T\right)$.

Replace the exact state-action value function $Q^{\pi_\theta}(s, a)$ with a Monte Carlo approximation G_t .

Recall $Q^{\pi_\theta}(s, a) = \mathbb{E}\left[G_t \mid S_t = s, \pi_\theta\right]$

Use $G_t \nabla \ln(\pi_\theta(A_t \mid S_t))$ as a stochastic estimate of $\nabla J(\theta)$.

$$\theta_{k+1} = \theta_k + \alpha G_t \nabla \ln(\pi_\theta(A_t \mid S_t))$$

The REINFORCE algorithm

$$\nabla J(\theta) \propto \mathbb{E}_{S\sim\mu,\, A\sim\pi_\theta(\cdot|S)} \left[Q^{\pi_\theta}(S, A) \nabla \ln(\pi_\theta(A \mid S)) \right]$$

The policy gradient theorem for the undiscounted, episodic case.

Note: we can't evaluate the r.h.s exactly, but we can estimate it from samples.

Idea: Rollout $\pi_\theta(s, a)$ in the environment to sample random trajectories $\tau = \left(S_0, A_0, R_0, S_1, A_1, R_1, \ldots, S_T\right)$.

Replace the exact state-action value function $Q^{\pi_\theta}(s, a)$ with a Monte Carlo approximation G_t .

Recall $Q^{\pi_\theta}(s, a) = \mathbb{E}\left[G_t \mid S_t = s, \pi_\theta\right]$

Use $G_t \nabla \ln(\pi_\theta(A_t \mid S_t))$ as a stochastic estimate of $\nabla J(\theta)$.

$$\theta_{k+1} = \theta_k + \alpha G_t \nabla \ln(\pi_\theta(A_t \mid S_t))$$

Note: constant of proportionality is absorbed into the learning rate α.

The REINFORCE algorithm

$$\nabla J(\theta) \propto \mathbb{E}_{S\sim\mu,\, A\sim\pi_\theta(\cdot|S)} \left[Q^{\pi_\theta}(S, A) \nabla \ln(\pi_\theta(A \mid S)) \right]$$

The policy gradient theorem for the undiscounted, episodic case.

Note: we can't evaluate the r.h.s exactly, but we can estimate it from samples.

Idea: Rollout $\pi_\theta(s, a)$ in the environment to sample random trajectories $\tau = \left(S_0, A_0, R_0, S_1, A_1, R_1, \ldots, S_T\right)$.

Replace the exact state-action value function $Q^{\pi_\theta}(s, a)$ with a Monte Carlo approximation G_t .

Recall $Q^{\pi_\theta}(s, a) = \mathbb{E}\left[G_t \mid S_t = s, \pi_\theta\right]$

Use $G_t \nabla \ln(\pi_\theta(A_t \mid S_t))$ as a stochastic estimate of $\nabla J(\theta)$.

$$\theta_{k+1} = \theta_k + \alpha G_t \nabla \ln(\pi_\theta(A_t \mid S_t))$$

Note: constant of proportionality is absorbed into the learning rate α.

REINFORCE: Monte-Carlo Policy-Gradient Control (episodic) for π_*

Input: a differentiable policy parameterization $\pi(a|s, \boldsymbol{\theta})$
Algorithm parameter: step size $\alpha > 0$
Initialize policy parameter $\boldsymbol{\theta} \in \mathbb{R}^{d'}$ (e.g., to $\mathbf{0}$)

Loop forever (for each episode):
  Generate an episode $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$, following $\pi(\cdot|\cdot, \boldsymbol{\theta})$
  Loop for each step of the episode $t = 0, 1, \ldots, T-1$:
    $G \leftarrow \sum_{k=t+1}^{T} \gamma^{k-t-1} R_k$ $\quad (G_t)$
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha \gamma^t G \nabla \ln \pi(A_t|S_t, \boldsymbol{\theta})$

Figure courtesy of Sutton and Barto (2018)

The REINFORCE algorithm

REINFORCE: Monte-Carlo Policy-Gradient Control (episodic) for π_*

Input: a differentiable policy parameterization $\pi(a|s, \boldsymbol{\theta})$
Algorithm parameter: step size $\alpha > 0$
Initialize policy parameter $\boldsymbol{\theta} \in \mathbb{R}^{d'}$ (e.g., to $\mathbf{0}$)

Loop forever (for each episode):
  Generate an episode $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$, following $\pi(\cdot|\cdot, \boldsymbol{\theta})$
  Loop for each step of the episode $t = 0, 1, \ldots, T-1$:
    $G \leftarrow \sum_{k=t+1}^{T} \gamma^{k-t-1} R_k$ (G_t)
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha \gamma^t G \nabla \ln \pi(A_t|S_t, \boldsymbol{\theta})$

Figure courtesy of Sutton and Barto (2018)

The REINFORCE algorithm

REINFORCE: Monte-Carlo Policy-Gradient Control (episodic) for π_*

Input: a differentiable policy parameterization $\pi(a|s,\boldsymbol{\theta})$
Algorithm parameter: step size $\alpha > 0$
Initialize policy parameter $\boldsymbol{\theta} \in \mathbb{R}^{d'}$ (e.g., to $\mathbf{0}$)

Loop forever (for each episode):
  Generate an episode $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$, following $\pi(\cdot|\cdot,\boldsymbol{\theta})$
  Loop for each step of the episode $t = 0, 1, \ldots, T-1$:
    $G \leftarrow \sum_{k=t+1}^{T} \gamma^{k-t-1} R_k$ $\qquad (G_t)$
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha \gamma^t G \nabla \ln \pi(A_t|S_t,\boldsymbol{\theta})$

Figure courtesy of Sutton and Barto (2018)

Results for simple corridor example.

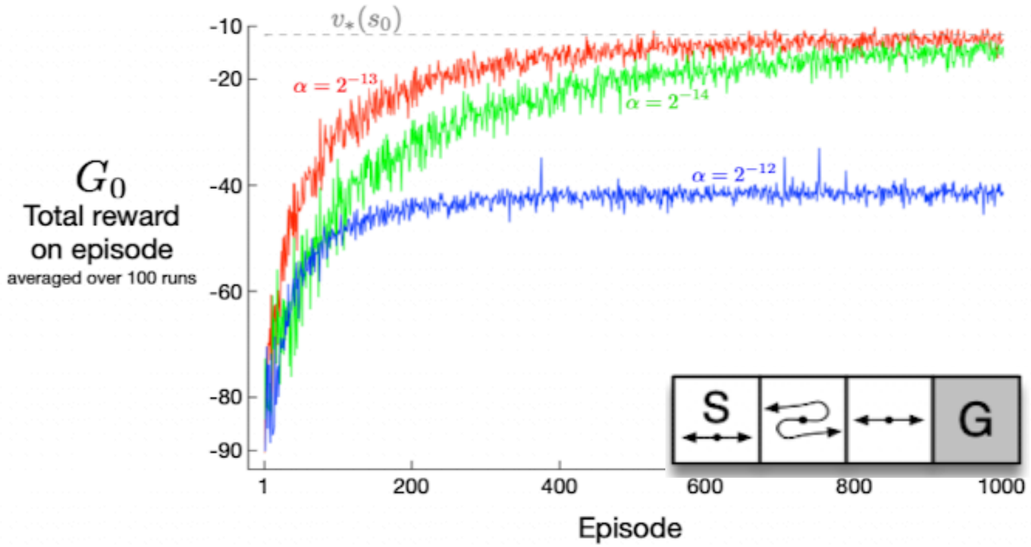


Figure courtesy of Sutton and Barto (2018)

A practical note on REINFORCE estimators

Note: In Sutton and Barto, the pseudocode makes a gradient update for every environment step t.

REINFORCE: Monte-Carlo Policy-Gradient Control (episodic) for π_*

Input: a differentiable policy parameterization $\pi(a|s, \boldsymbol{\theta})$
Algorithm parameter: step size $\alpha > 0$
Initialize policy parameter $\boldsymbol{\theta} \in \mathbb{R}^{d'}$ (e.g., to $\mathbf{0}$)

Loop forever (for each episode):
  Generate an episode $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$, following $\pi(\cdot|\cdot, \boldsymbol{\theta})$
  Loop for each step of the episode $t = 0, 1, \ldots, T-1$:
    $G \leftarrow \sum_{k=t+1}^{T} \gamma^{k-t-1} R_k$ (G_t)
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha \gamma^t G \nabla \ln \pi(A_t|S_t, \boldsymbol{\theta})$

Figure courtesy of Sutton and Barto (2018)

This is primarily for clarity of exposition.

In modern deep RL codes, we rarely update parameters after every environment step because:

- Estimator variance: a single trajectory gives a very noisy gradient estimate.
- Efficiency: It can be computationally more efficient to process a batch of trajectories together.

A practical note on REINFORCE estimators

Note: In Sutton and Barto, the pseudocode makes a gradient update for every environment step t.

REINFORCE: Monte-Carlo Policy-Gradient Control (episodic) for π_*

Input: a differentiable policy parameterization $\pi(a|s, \boldsymbol{\theta})$
Algorithm parameter: step size $\alpha > 0$
Initialize policy parameter $\boldsymbol{\theta} \in \mathbb{R}^{d'}$ (e.g., to $\mathbf{0}$)

Loop forever (for each episode):
  Generate an episode $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$, following $\pi(\cdot|\cdot, \boldsymbol{\theta})$
  Loop for each step of the episode $t = 0, 1, \ldots, T-1$:
    $G \leftarrow \sum_{k=t+1}^{T} \gamma^{k-t-1} R_k$ (G_t)
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha \gamma^t G \nabla \ln \pi(A_t|S_t, \boldsymbol{\theta})$

Figure courtesy of Sutton and Barto (2018)

This is primarily for clarity of exposition.

In modern deep RL codes, we rarely update parameters after every environment step because:

- Estimator variance: a single trajectory gives a very noisy gradient estimate.
- Efficiency: It can be computationally more efficient to process a batch of trajectories together.

Consider the true policy gradient $$\nabla J(\theta) \propto \mathbb{E}_{\tau \sim \pi_\theta, s_0 \sim \rho} \left[\sum_t G_t \nabla \ln(\pi_\theta(A_t \mid S_t)) \right]$$

Conceptually, we can define a per-trajectory REINFORCE gradient estimator $$\hat{g}(\tau) = \sum_t G_t \nabla \ln(\pi_\theta(A_t \mid S_t))$$

After collecting a batch of N trajectories in the environment, the Monte Carlo estimator we then use to define an individual gradient step would be given by

$$\hat{g}^N = \frac{1}{N} \sum_{i=1}^{N} \hat{g}(\tau) = \frac{1}{N} \sum_{i=1}^{N} \sum_t G_t \nabla \ln(\pi_\theta(A_t \mid S_t))$$

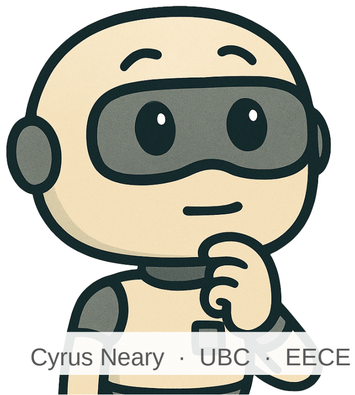
The REINFORCE algorithm

What are the strengths of the REINFORCE algorithm?

- Simplicity
- If G_t estimates are unbiased, the alg converges to a local optimum.

What are the weaknesses of the REINFORCE algorithm?

- We have to rollout entire trajectories just to get G_t
- If we don't have ALOT of samples, each estimate of $\nabla J(\theta)$ can have very high variance.



Baselines in policy gradient algorithms

Goal: Reduce the variance of stochastic policy gradient estimators.

Baselines in policy gradient algorithms

Goal: Reduce the variance of stochastic policy gradient estimators.

Consider the true policy gradient $\nabla J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta, s_0 \sim \rho} \left[\sum_t G_t \nabla \ln(\pi_\theta(A_t \mid S_t)) \right]$

and the per-trajectory REINFORCE gradient estimator $\hat{g}(\tau) = \sum_t G_t \nabla \ln(\pi_\theta(A_t \mid S_t))\,.$

Baselines in policy gradient algorithms

Goal: Reduce the variance of stochastic policy gradient estimators.

Consider the true policy gradient $\nabla J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta, s_0 \sim \rho} \left[\sum_t G_t \nabla \ln(\pi_\theta(A_t \mid S_t)) \right]$

and the per-trajectory REINFORCE gradient estimator $\hat{g}(\tau) = \sum_t G_t \nabla \ln(\pi_\theta(A_t \mid S_t))\,.$

Define a state-dependent *baseline* function $b(s)$, and define an updated per-trajectory estimator $\hat{g}_b(\tau)$:

$$\hat{g}_b(\tau) = \sum_t \big(G_t - b(S_t)\big) \nabla \ln(\pi_\theta(A_t \mid S_t))$$

Baselines in policy gradient algorithms

Goal: Reduce the variance of stochastic policy gradient estimators.

Consider the true policy gradient $\nabla J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta, s_0 \sim \rho} \left[\sum_t G_t \nabla \ln(\pi_\theta(A_t \mid S_t)) \right]$

and the per-trajectory REINFORCE gradient estimator $\hat{g}(\tau) = \sum_t G_t \nabla \ln(\pi_\theta(A_t \mid S_t))\,.$

Define a state-dependent *baseline* function $b(s)$, and define an updated per-trajectory estimator $\hat{g}_b(\tau)$:

$$\hat{g}_b(\tau) = \sum_t \left(G_t - b(S_t) \right) \nabla \ln(\pi_\theta(A_t \mid S_t))$$

Claim: For the right choice of baseline function $b(s)$, $\hat{g}_b(\tau)$ is an unbiased estimator with a lower variance than $\hat{g}(\tau)$.

i.e., $\mathbb{E}_{\tau \sim \pi_\theta, s_0 \sim \rho} \left[\hat{g}_b(s) \right] = \mathbb{E}_{\tau \sim \pi_\theta, s_0 \sim \rho} \left[\hat{g}(\tau) \right] = \nabla J(\theta)$ and, $Var \left[\hat{g}_b(s) \right] < Var \left[\hat{g}(\tau) \right]$

Baselines in policy gradient algorithms, continued

Claim: For the right choice of baseline function $b(s)$, $\hat{g}_b(\tau)$ is an unbiased estimator with a lower variance than $\hat{g}(\tau)$.

i.e., $\mathbb{E}_{\tau\sim\pi_\theta, s_0\sim\rho}\left[\hat{g}_b(s)\right] = \mathbb{E}_{\tau\sim\pi_\theta, s_0\sim\rho}\left[\hat{g}(\tau)\right] = \nabla J(\theta)$ and, $Var\left[\hat{g}_b(s)\right] < Var\left[\hat{g}(\tau)\right]$

REINFORCE with baseline

Idea: Learn an estimate of $V_w^{\pi_\theta}(s)$ of the state value function for the policy π_θ, with parameter vector $w \in \mathbb{R}^{d_w}$.

Train $V_w^{\pi_\theta}(s)$ using Monte Carlo estimates of return G_t.

Use this value estimate as a baseline function.

$$\theta_{k+1} = \theta_k + \alpha \left(G_t - V_w^{\pi_\theta}(S_t)\right) \nabla \ln \left(\pi_\theta(A_t \mid S_t)\right)$$

monte carlo baseline

REINFORCE with Baseline (episodic), for estimating $\pi_\theta \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a|s,\boldsymbol{\theta})$
Input: a differentiable state-value function parameterization $\hat{v}(s,\mathbf{w})$
Algorithm parameters: step sizes $\alpha^{\boldsymbol{\theta}} > 0$, $\alpha^{\mathbf{w}} > 0$
Initialize policy parameter $\boldsymbol{\theta} \in \mathbb{R}^{d'}$ and state-value weights $\mathbf{w} \in \mathbb{R}^{d}$ (e.g., to $\mathbf{0}$)

Loop forever (for each episode):
  Generate an episode $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$, following $\pi(\cdot|\cdot,\boldsymbol{\theta})$
  Loop for each step of the episode $t = 0, 1, \ldots, T-1$:
    $G \leftarrow \sum_{k=t+1}^{T} \gamma^{k-t-1} R_k$ (G_t)
    $\delta \leftarrow G - \hat{v}(S_t,\mathbf{w})$
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha^{\mathbf{w}} \delta \nabla \hat{v}(S_t,\mathbf{w})$
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha^{\boldsymbol{\theta}} \gamma^t \delta \nabla \ln \pi(A_t|S_t,\boldsymbol{\theta})$

Figure courtesy of Sutton and Barto (2018)

REINFORCE with baseline

Idea: Learn an estimate of $V_w^{\pi_\theta}(s)$ of the state value function for the policy π_θ, with parameter vector $w \in \mathbb{R}^{d_w}$.

Train $V_w^{\pi_\theta}(s)$ using Monte Carlo estimates of return G_t.

Use this value estimate as a baseline function.

$$\theta_{k+1} = \theta_k + \alpha \left(G_t - V_w^{\pi_\theta}(S_t)\right) \nabla \ln \left(\pi_\theta(A_t \mid S_t)\right)$$

REINFORCE with Baseline (episodic), for estimating $\pi_\theta \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a|s,\boldsymbol{\theta})$
Input: a differentiable state-value function parameterization $\hat{v}(s,\mathbf{w})$
Algorithm parameters: step sizes $\alpha^{\boldsymbol{\theta}} > 0$, $\alpha^{\mathbf{w}} > 0$
Initialize policy parameter $\boldsymbol{\theta} \in \mathbb{R}^{d'}$ and state-value weights $\mathbf{w} \in \mathbb{R}^d$ (e.g., to $\mathbf{0}$)

Loop forever (for each episode):
  Generate an episode $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$, following $\pi(\cdot|\cdot,\boldsymbol{\theta})$
  Loop for each step of the episode $t = 0, 1, \ldots, T-1$:
    $G \leftarrow \sum_{k=t+1}^{T} \gamma^{k-t-1} R_k$ (G_t)
    $\delta \leftarrow G - \hat{v}(S_t,\mathbf{w})$
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha^{\mathbf{w}} \delta \nabla \hat{v}(S_t,\mathbf{w})$
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha^{\boldsymbol{\theta}} \gamma^t \delta \nabla \ln \pi(A_t|S_t,\boldsymbol{\theta})$

Figure courtesy of Sutton and Barto (2018)

Results for simple corridor example.

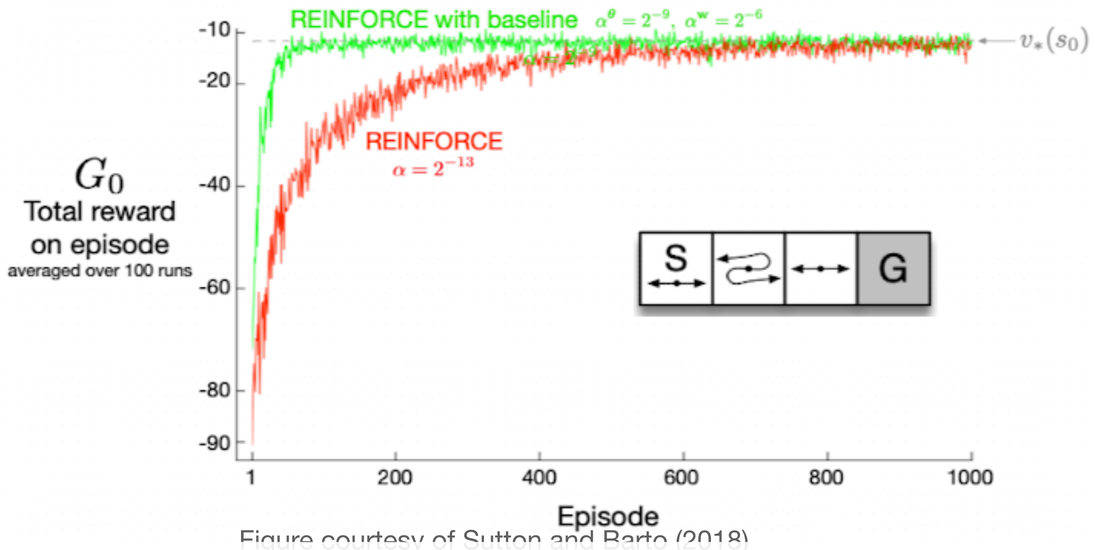


Figure courtesy of Sutton and Barto (2018)

REINFORCE with baseline

Idea: Learn an estimate of $V_w^{\pi_\theta}(s)$ of the state value function for the policy π_θ, with parameter vector $w \in \mathbb{R}^{d_w}$.

Train $V_w^{\pi_\theta}(s)$ using Monte Carlo estimates of return G_t.

Use this value estimate as a baseline function.

$$\theta_{k+1} = \theta_k + \alpha \left(G_t - V_w^{\pi_\theta}(S_t)\right) \nabla \ln \left(\pi_\theta(A_t \mid S_t)\right)$$

REINFORCE with Baseline (episodic), for estimating $\pi_\theta \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a|s,\boldsymbol{\theta})$
Input: a differentiable state-value function parameterization $\hat{v}(s,\mathbf{w})$
Algorithm parameters: step sizes $\alpha^{\boldsymbol{\theta}} > 0$, $\alpha^{\mathbf{w}} > 0$
Initialize policy parameter $\boldsymbol{\theta} \in \mathbb{R}^{d'}$ and state-value weights $\mathbf{w} \in \mathbb{R}^d$ (e.g., to $\mathbf{0}$)

Loop forever (for each episode):
  Generate an episode $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$, following $\pi(\cdot|\cdot,\boldsymbol{\theta})$
  Loop for each step of the episode $t = 0, 1, \ldots, T-1$:
    $G \leftarrow \sum_{k=t+1}^{T} \gamma^{k-t-1} R_k$ $\quad (G_t)$
    $\delta \leftarrow G - \hat{v}(S_t,\mathbf{w})$
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha^{\mathbf{w}} \delta \nabla \hat{v}(S_t,\mathbf{w})$
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha^{\boldsymbol{\theta}} \gamma^t \delta \nabla \ln \pi(A_t|S_t,\boldsymbol{\theta})$

Figure courtesy of Sutton and Barto (2018)

Results for simple corridor example.

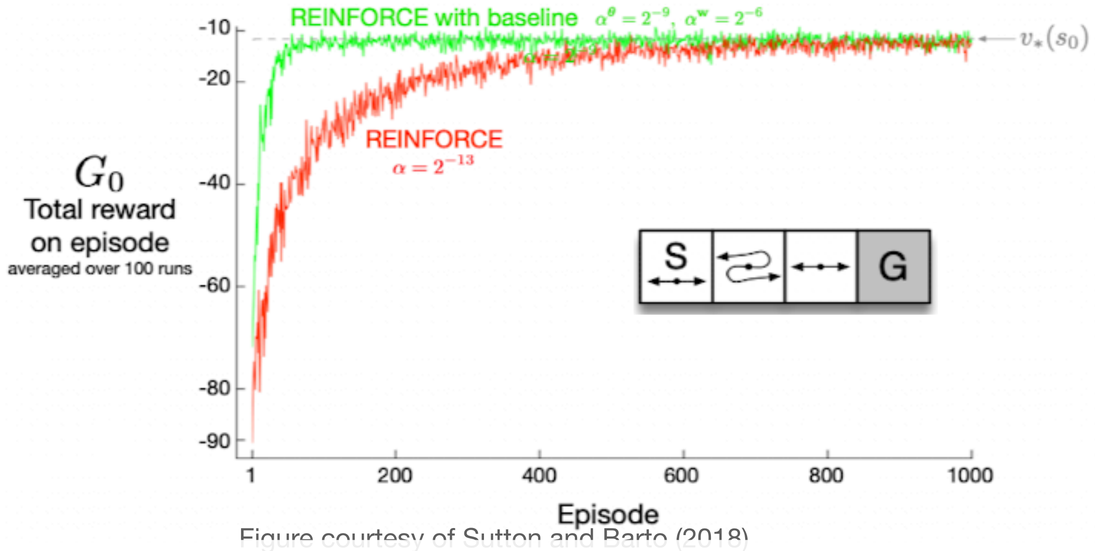


Figure courtesy of Sutton and Barto (2018)

- Adding a baseline can make learning much faster.
- But, it requires learning an extra function $V_w^{\pi_\theta}(s)$.
- And, it requires tuning an extra hyperparameter:
  - One learning rate α_θ for policy parameters $\theta \in \mathbb{R}^{d_\theta}$.
  - One learning rate α_w for value function parameters $w \in \mathbb{R}^{d_w}$.

An intuition on the form of the policy gradient

$$\nabla J(\theta) \propto \mathbb{E}_{S\sim\mu,\, A\sim\pi_\theta(\cdot|S)} \left[Q^{\pi_\theta}(S, A) \nabla \ln(\pi_\theta(A \mid S)) \right]$$

An intuition on the form of the policy gradient

$$\nabla J(\theta) \propto \mathbb{E}_{S\sim\mu,\, A\sim\pi_\theta(\cdot|S)} \left[Q^{\pi_\theta}(S, A) \nabla \ln(\pi_\theta(A \mid S)) \right]$$

For an action $a \sim \pi_\theta(\cdot \mid s)$ sampled from state $s \sim \mu$…

- How “good” was the action?

An intuition on the form of the policy gradient

$$\nabla J(\theta) \propto \mathbb{E}_{S\sim\mu,\, A\sim\pi_\theta(\cdot|S)} \left[Q^{\pi_\theta}(S, A) \nabla \ln(\pi_\theta(A \mid S)) \right]$$

For an action $a \sim \pi_\theta(\cdot \mid s)$ sampled from state $s \sim \mu$…

- How “good” was the action?
- What direction in parameter space $\theta \in \mathbb{R}^{d_\theta}$ increases the probability of $\pi_\theta(a \mid s)$ the fastest?

An intuition on the form of the policy gradient

$$\nabla J(\theta) \propto \mathbb{E}_{S\sim\mu,\, A\sim\pi_\theta(\cdot|S)} \left[Q^{\pi_\theta}(S,A)\, \nabla \ln(\pi_\theta(A \mid S)) \right]$$

For an action $a \sim \pi_\theta(\cdot \mid s)$ sampled from state $s \sim \mu$…

- How “good” was the action?
- What direction in parameter space $\theta \in \mathbb{R}^{d_\theta}$ increases the probability of $\pi_\theta(a \mid s)$ the fastest?

Their product can intuitively be interpreted as:

(**Critique of the action**) x (**Which direction to nudge the policy**)

An intuition on the form of the policy gradient

$$\nabla J(\theta) \propto \mathbb{E}_{S\sim\mu,\, A\sim\pi_\theta(\cdot|S)} \left[Q^{\pi_\theta}(S,A)\, \nabla \ln(\pi_\theta(A \mid S)) \right]$$

For an action $a \sim \pi_\theta(\cdot \mid s)$ sampled from state $s \sim \mu$…

- How “good” was the action?
- What direction in parameter space $\theta \in \mathbb{R}^{d_\theta}$ increases the probability of $\pi_\theta(a \mid s)$ the fastest?

Their product can intuitively be interpreted as:

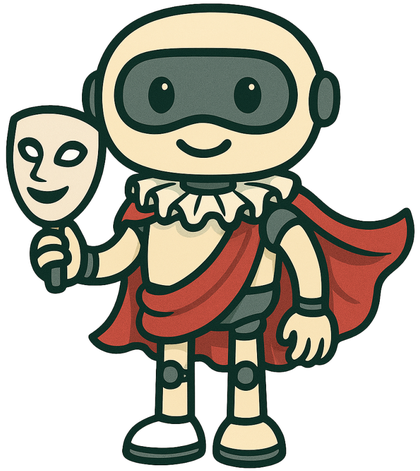
(**Critique of the action**) x (**Which direction to nudge the policy**)

Actions with higher values will contribute more to the overall policy gradient.

Actor-Critic methods

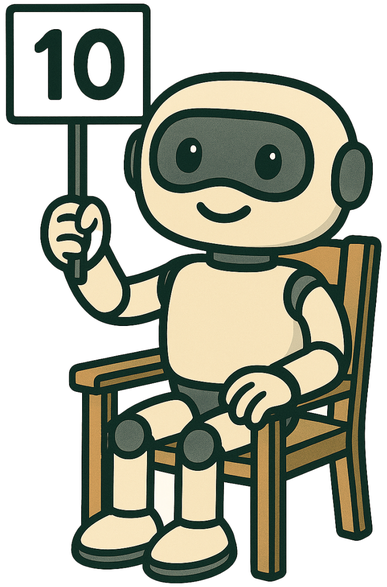
Generally speaking, RL algorithms that combine:

- An **actor**: the parameterized policy $\pi_\theta(a \mid s)$.
- A **critic**: an approximation of a value function $V_w(s)$ or $Q_w(s, a)$ that evaluates how good the actor's choices are.



Actor $\pi_\theta(a \mid s)$

Takes actions in the environment, given the current state.



Critic $V_w(s)$

Evaluates the quality of the actor's chosen actions. Used to improve the actor's decisions during training.

Both the actor and the critic are learned simultaneously as the learning algorithm progresses.

Actor-Critic methods

In REINFORCE with baseline, we learned $\pi_\theta(a \mid s)$ and $V_w^{\pi_\theta}(s)$.

$$\theta_{k+1} = \theta_k + \alpha \left(G_t - V_w^{\pi_\theta}(S_t)\right) \nabla \ln \left(\pi_\theta(A_t \mid S_t)\right)$$

Note, however, this sometimes isn't considered as an "Actor-Critic" algorithm, as we are using the learned value function as a baseline, instead of for bootstrapping, or to play the role of the "critic" directly.

Actor-Critic methods

In REINFORCE with baseline, we learned $\pi_\theta(a \mid s)$ and $V_w^{\pi_\theta}(s)$.

$$\theta_{k+1} = \theta_k + \alpha \left(G_t - V_w^{\pi_\theta}(S_t)\right) \nabla \ln \left(\pi_\theta(A_t \mid S_t)\right)$$

Note, however, this sometimes isn't considered as an "Actor-Critic" algorithm, as we are using the learned value function as a baseline, instead of for bootstrapping, or to play the role of the "critic" directly.

Instead, we could use $V_w^{\pi_\theta}(s)$ to define a TD(0) approximation of G_t.

I.e., we are using a bootstrapped estimate of the return, to help define our policy gradient to improve the actor!

$$\theta_{k+1} = \theta_k + \alpha \left(R_t + \gamma V_w^{\pi_\theta}(S_{t+1}) - V_w^{\pi_\theta}(S_t)\right) \nabla \ln \left(\pi_\theta(A_t \mid S_t)\right)$$

Actor-Critic methods

In REINFORCE with baseline, we learned $\pi_\theta(a \mid s)$ and $V_w^{\pi_\theta}(s)$.

$$\theta_{k+1} = \theta_k + \alpha \left(G_t - V_w^{\pi_\theta}(S_t)\right) \nabla \ln \left(\pi_\theta(A_t \mid S_t)\right)$$

Note, however, this sometimes isn't considered as an "Actor-Critic" algorithm, as we are using the learned value function as a baseline, instead of for bootstrapping, or to play the role of the "critic" directly.

Instead, we could use $V_w^{\pi_\theta}(s)$ to define a TD(0) approximation of G_t.

I.e., we are using a bootstrapped estimate of the return, to help define our policy gradient to improve the actor!

$$\theta_{k+1} = \theta_k + \alpha \left(R_t + \gamma V_w^{\pi_\theta}(S_{t+1}) - V_w^{\pi_\theta}(S_t)\right) \nabla \ln \left(\pi_\theta(A_t \mid S_t)\right)$$

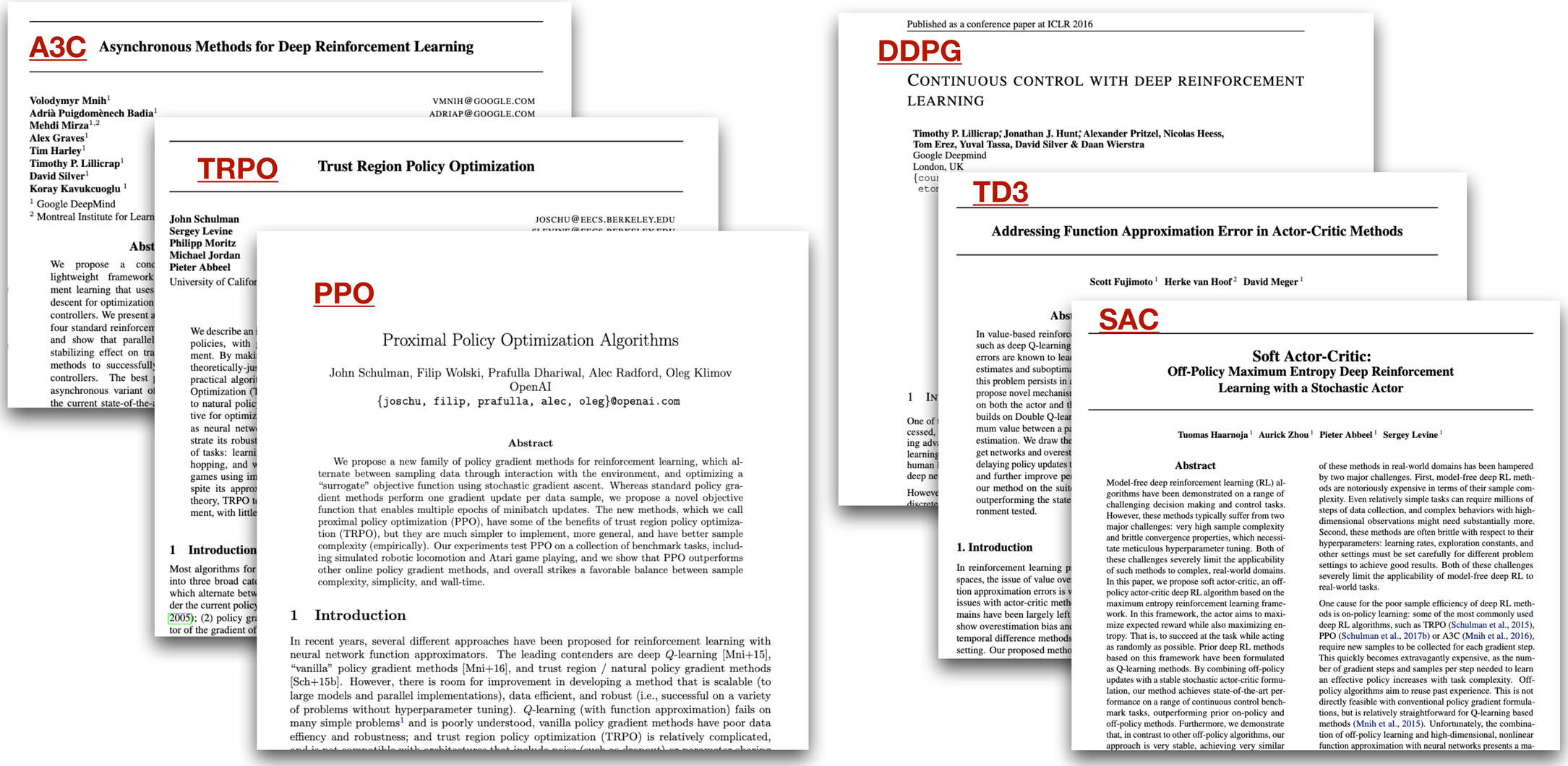
The result: a one-step episodic Actor-Critic method.

One-step Actor–Critic (episodic), for estimating $\pi_\theta \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a|s,\boldsymbol{\theta})$
Input: a differentiable state-value function parameterization $\hat{v}(s,\mathbf{w})$
Parameters: step sizes $\alpha^{\boldsymbol{\theta}} > 0$, $\alpha^{\mathbf{w}} > 0$
Initialize policy parameter $\boldsymbol{\theta} \in \mathbb{R}^{d'}$ and state-value weights $\mathbf{w} \in \mathbb{R}^d$ (e.g., to $\mathbf{0}$)
Loop forever (for each episode):
  Initialize S (first state of episode)
  $I \leftarrow 1$
  Loop while S is not terminal (for each time step):
    $A \sim \pi(\cdot|S,\boldsymbol{\theta})$
    Take action A, observe S', R
    $\delta \leftarrow R + \gamma \hat{v}(S',\mathbf{w}) - \hat{v}(S,\mathbf{w})$ (if S' is terminal, then $\hat{v}(S',\mathbf{w}) \doteq 0$)
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha^{\mathbf{w}} \delta \nabla \hat{v}(S,\mathbf{w})$
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha^{\boldsymbol{\theta}} I \delta \nabla \ln \pi(A|S,\boldsymbol{\theta})$
    $I \leftarrow \gamma I$
    $S \leftarrow S'$

Actor-Critic methods

More broadly, Actor-Critic methods are very important and common in modern deep RL algorithms.



More on these algorithms in the next few lectures!